

Day 6: Debug Loss Transition

Goal

Make CICFM failures local and explainable. The transition is considered debuggable when the runtime can answer these questions for every step:

```
Did the MSE baseline pass before CICFM started?  
Was the CICFM batch assembled correctly?  
Is the model using the CICFM input dimension?  
Is the loss comparing predicted_velocity to target_velocity?  
Are loss, outputs, and gradients finite?  
Is the first CICFM behavior reasonable before optimizer updates?
```

Do not tune learning rate, architecture, Fourier feature count, or random sampling until these checks pass.

Required Precondition

Start Day 6 only after Day 5 gates pass:

```
MSE tiny fixed-batch loss decreases  
MSE gradients are finite  
CICFM deterministic batch assembly passes  
CICFM one-step forward loss is finite  
CICFM one-step backward gradients are finite
```

If any of these fail, return to the relevant Day 4 or Day 5 check before running CICFM training.

Debug Run Order

Run the transition in this exact order:

1. MSE deterministic overfit, $F = 8$.
2. MSE random sampled batch, $F = 8$.
3. CICFM deterministic assembly, $z_0 = 0$, $t = 0.5$, no optimizer step.
4. CICFM deterministic forward and backward, no optimizer step.
5. CICFM deterministic overfit for a tiny fixed batch.
6. CICFM random source/time batch, $F = 8$.
7. CICFM random source/time batch, $F = 16$.

Do not move to the next stage until the current stage has finite loss, finite gradients, and stable tensor statistics.

Deterministic CICFM Probe

Use this probe before every random CICFM debugging session:

```
z0 = zeros_like(targets)
z1 = targets
t = 0.5
zt = 0.5 * targets
target_velocity = targets
cicfm_input = concat(features, zt, t)
```

Expected values:

```
min_t = 0.5
max_t = 0.5
mean_t = 0.5
max_abs(zt - 0.5 * targets) < 1e-6
max_abs(target_velocity - targets) < 1e-6
cicfm_input_dim = feature_dim + C + 1
```

For the RGB image baseline:

```
F = 8:  cicfm_input_dim = 38
F = 16: cicfm_input_dim = 70
output_dim = 3
```

If this probe fails, the bug is in CICFM assembly or shape dispatch, not in the optimizer.

Loss Target Probe

Use a small hand-checkable batch to prove the loss target is velocity:

```
predicted_velocity = [[[0.0, 0.0, 0.0]]]
z0 = [[[0.2, 0.4, 0.6]]]
z1 = [[[0.5, 0.1, 0.9]]]
target_velocity = z1 - z0 = [[[0.3, -0.3, 0.3]]]
```

Expected CICFM loss:

```
loss = mean((predicted_velocity - target_velocity)^2)
      = (0.09 + 0.09 + 0.09) / 3
```

```
= 0.09
```

Expected output gradient:

```
d_predicted_velocity =  
    2 * (predicted_velocity - target_velocity) / 3  
    = [[[-0.2, 0.2, -0.2]]]
```

If the loss is instead:

```
mean((predicted_velocity - z1)^2)
```

the implementation is still comparing against targets instead of velocity.

Required Debug Statistics

Log these fields for every Day 6 training step:

```
step  
loss_mode  
loss  
batch_size  
num_points  
feature_dim  
input_dim  
output_dim  
learning_rate  
mean_abs_output  
mean_abs_loss_target  
mean_abs_output_gradient  
max_abs_output_gradient  
all_outputs_finite  
all_gradients_finite
```

For MSE, interpret:

```
output = prediction  
loss_target = targets  
output_gradient = d_prediction
```

For CICFM, interpret:

```
output = predicted_velocity
loss_target = target_velocity
output_gradient = d_predicted_velocity
```

Add these CICFM-only fields:

```
min_t
max_t
mean_t
mean_abs_z0
mean_abs_z1
mean_abs_zt
mean_abs_target_velocity
mean_abs_predicted_velocity
max_abs_target_velocity
```

Finite-Value Policy

Abort the current training run immediately if any of these are false:

```
loss is finite
all model outputs are finite
all output gradients are finite
all parameter gradients are finite
all parameter values are finite after optimizer_step()
```

On abort, print:

```
loss_mode
step
input_dim
output_dim
learning_rate
first failing tensor name
first failing tensor index
first failing tensor value
```

The first failing tensor is more useful than a later NaN in the loss.

Expected Loss Behavior

For the deterministic CICFM overfit run:

```
z0 = 0
t = 0.5
target_velocity = targets
```

The problem is intentionally close to MSE overfit. Expected behavior:

```
initial loss is finite
loss decreases over the first short run
mean_abs_predicted_velocity moves toward mean_abs_target_velocity
all gradients remain finite
```

For random CICFM:

```
z0 ~ Uniform(0, 1)
t ~ Uniform(0, 1)
target_velocity = z1 - z0
```

Expected behavior:

```
loss may be noisier than MSE
loss should remain finite
mean_t should stay near 0.5 over large enough batches
min_t and max_t should remain inside [0, 1]
target_velocity magnitude should be consistent with z1 - z0
```

Do not require random CICFM loss to decrease smoothly step by step. Require a downward trend over a fixed window after deterministic CICFM passes.

First Debug Window

Use a small fixed window before long training:

```
batch_size = 1
num_points = 64
F = 8
hidden_width = 256 for the real baseline
steps = 100
learning_rate = the MSE overfit learning rate first
```

If this is too slow for finite-difference or verbose debugging, use a separate debug model:

```
input_dim = feature_dim + C + 1
hidden_width = 4
output_dim = 3
steps = 20
```

The tiny debug model exists only for math checks. It must not replace the baseline architecture.

Shape Assertions

Check these before the forward pass:

```
features.shape == [B, N, feature_dim]
z0.shape == [B, N, C]
z1.shape == [B, N, C]
t.shape == [B, N, 1]
zt.shape == [B, N, C]
target_velocity.shape == [B, N, C]
cicfm_input.shape == [B, N, feature_dim + C + 1]
```

Check these after the forward pass:

```
predicted_velocity.shape == [B, N, C]
loss_target.shape == [B, N, C]
d_predicted_velocity.shape == [B, N, C]
```

Flattening must preserve the same row meaning:

```
row = b * N + n
X[row] corresponds to target_velocity[b, n]
```

Debug Triage Table

Symptom	Likely Cause	First Check
CICFM loss equals MSE loss exactly	Still using targets as loss target	Run loss target probe
Shape mismatch in first layer	MSE model instance reused	Check input_dim and linear0.weight
Loss is finite but does not move	Output target dispatch is wrong or learning rate too low	Compare mean_abs_output and mean_abs_loss_target

Symptom	Likely Cause	First Check
NaN after optimizer step	Learning rate too high or gradient explosion	Check <code>max_abs_output_gradient</code> before step
<code>mean_t</code> is always 0 or 1	Time sampler or deterministic flag stuck	Check <code>min_t</code> , <code>max_t</code> , <code>mean_t</code>
<code>target_velocity</code> changes with <code>t</code>	Velocity formula incorrectly uses interpolation	Check <code>target_velocity = z1 - z0</code>
<code>zt</code> outside value range	Interpolation or source range bug	Check <code>z0</code> , <code>z1</code> , <code>t</code> , <code>zt</code>
CUDA parity fails only in CICFM	CUDA embedding path omits raw coordinate or extra CICFM fields	Check feature concat boundary

CUDA Alignment Check

Day 6 does not require CUDA model training, but the embedding path must stay aligned with the CICFM input contract:

```
features = concat(raw_coordinates, fourier_features)
cicfm_input = concat(features, zt, t)
```

The CUDA embedding kernel currently covers the Fourier-feature portion. Runtime assembly must verify that raw coordinates, `zt`, and `t` are appended in the same order on CPU and CUDA-backed paths.

Day 6 Definition of Done

Day 6 is complete when:

1. The deterministic CICFM probe is documented and required before random runs.
2. The loss target probe proves CICFM compares against `target_velocity`.
3. Required debug logging fields are frozen for MSE and CICFM.
4. Finite-value abort policy is defined.
5. Expected deterministic and random CICFM loss behavior is defined.
6. Shape assertions cover pre-forward, post-forward, and flattening alignment.
7. The triage table maps common failure symptoms to first checks.
8. CUDA embedding alignment is stated without expanding Day 6 into CUDA model training.